

Subjects to crack MSc in CSE BUET Admission Test

- | | |
|---|--|
| 1. Structured Programming Language | 11. Information System Design |
| 2. Discrete Mathematics | 12. Microprocessors and Microcontrollers |
| 3. Object Oriented Programming Language | 13. Computer Architecture |
| 4. Data Structures | 14. Operating Systems |
| 5. Digital logic Design | 15. Artificial Intelligence |
| 6. Algorithms | 16. Data Communication |
| 7. Compiler | 17. Automata Theory |
| 8. Database | 18. Computer Security |
| 9. Computer Network | 19. Machine Learning |
| 10. Software Engineering | 20. Computer Graphics |

This list changes every now and then. So, focus on Current syllabus.

Admission Test Type

Written:

Total question: 30

Each question carries: 5 points (Total $5 \times 30 = 150$)

Time: 1 hour 30 minutes

Use of scientific calculator is not prohibited but not encouraged either. It depends on the examiner. So the best idea is to learn math's without calculator. Take your calculator with you in exam hall. Understand the situation and act accordingly.

Viva:

For Viva voce you need to prepare for basic questions from CSE life. They can ask some additional things as follows:

1. Your job experience (if any). Whether you will be able to handle the pressure or not. Because, to maintain Job and MSc in CSE BUET is so difficult that majority of them gets stuck and extends semester after semester.
2. Part time/ full time preference.
3. Your project work in bachelor.
4. Your family background and willingness of admission at BUET.
5. They may ask your opinion on current affairs.

The following information is collected from BUET official Site. Information can be modified in future. So please follow latest updated information from their website.

Prerequisite For M.Sc. & M.Engg.:

For admission to the courses leading to a M.Sc. & M.Engg. degree a candidate:

1. ☐ Must have at least two first class/first divisions in public examinations.
2. ☐ Should have at least 50% marks or GPA of a minimum of 2.50 out of 4.0 or its equivalent in four-years B.A or B.Sc. Degree / B.Sc. Engg. in the relevant branch
3. ☐ Must not have any third division / class in any public examination.

Important Information

Upon submission of the online application through the website of BUET <http://www.buet.ac.bd> the printed copy of the completed Application Form together with the following supporting documents must be submitted to the Registrar's Office, BUET within the period during office hours (9:00 am to 5:00 pm) to complete the application procedure.

1. Cash payment of Tk. 500/- (Five Hundred) as application fee (non-refundable).
2. Two attested copies of recent passport-size color photographs (applicant's name must be written on the reverse side of each photograph).
3. Attested copies of certificates of all public examinations.
4. Attested copies of transcripts/ grade-sheets/ mark-sheets of degrees obtained.
5. Attested copies of testimonial character certificate from the institution last attended.
6. Letter of Consent, in prescribed form, from the employer for applicants in employment.

Prospectus containing detailed qualification requirements for admission into different programs and information regarding the submission of application is available at <http://www.buet.ac.bd>

Applicants in employment whether part-time or full-time must apply through their employers for both part-time and full-time enrollments. Incomplete applications (submitted online but no hardcopy of the Application Form together with all supporting documents submitted to Registrar's Office, BUET) will not be accepted. Applications submitted by mail will not be accepted. Selection of candidates for admission into a postgraduate program is performed by respective department/ institute. The selection procedure may vary from one department to another. Candidates are advised to contact the respective department institute for information regarding the selection procedure. A few Teaching Assistantship/Fellowships may be awarded to suitable full-time students. Degrees obtained from institutions other than BUET will be subjected to equivalence by appropriate authority of BUET before a candidate is finally selected for admission to a degree/diploma program.

Academic Requirements and Regulations:

- ▶ The minimum duration of the M. Sc. Engg. / M. Engg. course shall normally be three semesters. A candidate for the Master's degree must complete all the requirements for the degree within five academic years (session) from the date of the first admission in the respective program.
- ▶ Academic progress shall be measured in terms of credit hours earned by a student. One credit hour subject shall normally require one hour of class attendance per week for one semester; while one credit hour of thesis/project/laboratory should normally require three hours of work per week for one semester. The number of credit hours for each subject shall be as specified in the syllabus.
- ▶ For the degree of M. Sc. Engg. a student must earn a minimum of 36 credit hours including a thesis for which 18 credit hours shall be assigned.
- ▶ For the degree of M. Engg. a student must earn a minimum of 36 credit hours including a project for which 6 credit hours shall be assigned.
- ▶ There shall be two categories of student, namely, full-time students and part-time students.
- ▶ Students, serving in different organizations, may be admitted as part time students with a written consent of the employer. A part time student may be assigned a maximum of 9 credit hours of course including thesis / project work in any semester.
- ▶ Full-time students must register for a minimum of 12 credit hours and a maximum of 15 credit hours per semester. A full-time student shall not be allowed to be in the employment of any organization (even as a part time employee). However, they may be employed as teaching / research assistant at the University. If a full-time student becomes an employee (full-time or part-time) of any other organization in the middle of a semester, he may, with the approval of the Head of the department and his employer, be allowed to continue as a full time student for that semester.
- ▶ The courses of study is recommended by the BPGS and Faculty and approved by the Academic Council. The BPGS may review the curriculum from time to time and recommend any changes as may be considered necessary. The courses to be offered in any semester shall also be as determined by the BPGS.
- ▶ A student on recommendation of the BPGS and as approved by the CASR may be allowed a transfer of credits of the courses completed by the student at a recognized institution provided that the courses were not taken earlier than five calendar years from the date of his first enrollment in the respective program at BUET and that the student obtained a minimum GPA of 3.0 out of 4.0 or its equivalent and that the courses are equivalent to the approved courses of BUET.

Detail Syllabus

Discrete Mathematics

Set theory; Relations; Functions; Graph theory; Propositional calculus and predicate calculus; Mathematical reasoning: induction, contradiction and recursion; counting; Principles of inclusion and exclusion; Recurrence relations; Algebraic structures: rings and groups.

Structured Programming Language

Structured programming language: data types, operators, expressions, control structures; Functions and program structure: parameter passing conventions, scope rules and storage classes, recursion; Header files; Preprocessor; Pointers and arrays; Strings; Multidimensional array; User defined data types: structures, unions, enumerations; Input and Output: standard input and output, formatted input and output, file access; Variable length argument list; Command line parameters; Error Handling; Graphics; Linking; Library functions. Reference language: C

Object Oriented Programming Language

Philosophy of Object-Oriented Programming (OOP); Advantages of OOP over structured programming; Encapsulation, classes and objects, access specifiers, static and non-static members; Constructors, destructors and copy constructors; Array of objects, object pointers, and object references; Inheritance: single and multiple inheritance; Polymorphism: overloading, abstract classes, virtual functions and overriding; Exceptions; Object Oriented I/O; Template functions and classes; Multi-threaded Programming.

Reference languages: C++ and Java.

Data Structures

Internal data representation; Abstract data types; Elementary data structures: arrays, lists, stacks, queues, trees, graphs; Advanced data Structures: heaps, Fibonacci heaps, B-trees; Recursion, sorting, searching, hashing, storage management.

Compiler

Introduction to compiling; Basic issues; Lexical analysis; Syntax analysis; Syntax directed translation; Semantic analysis: type-checking; Run-time environments; Intermediate code generation; Code generation; Code optimization.

Digital Logic Design

Digital logic: Boolean algebra, De Morgan's Theorems, logic gates and their truth tables, canonical forms, combinational logic circuits, minimization techniques; Arithmetic and data handling logic circuits, decoders and encoders, multiplexers and demultiplexers; Combinational circuit design; Flip-flops, race around problems; Counters: asynchronous counters, synchronous counters and their applications; PLA design; Synchronous and asynchronous logic design; State diagram, Mealy and Moore machines; State minimizations and assignments; Pulse mode logic; Fundamental mode design.

Algorithms

Techniques for analysis of algorithms; Methods for the design of efficient algorithms: divide and conquer, greedy method, dynamic programming, back tracking, branch and bound; Basic search and traversal techniques; Topological sorting; Connected components, spanning trees, shortest paths; Flow algorithms; Approximation algorithms; Parallel algorithms; Algebraic simplification and transformations; Lower bound theory; NP-completeness, NP-hard and NP-complete problems.

Digital Electronics

Diode logic gates, transistor switches, transistor gates, MOS gates; Logic Families: TTL, ECL, IIL and CMOS logic with operation details; Propagation delay, product and noise immunity; Open collector and high impedance gates; Electronic circuits for flip-flops, counters and register, memory systems, PLAs; A/D and D/A converters with applications; S/H circuits, LED, LCD and optically coupled oscillators; Non-linear applications of OP AMPs; Analog switches.

Theory of Computation

Language theory; Finite automata: deterministic finite automata, nondeterministic finite automata, equivalence and conversion of deterministic and nondeterministic finite automata, pushdown automata; Context free languages; Context free grammars; Turing Machines: basic machines, configuration, computing with Turing machines, combining Turing machines; Undecidability.

Microprocessors and Microcontrollers

Introduction to 8-bit, 16-bit, and 32-bit microprocessors: architecture, addressing modes, instruction set, interrupts, multi-tasking and virtual memory; Memory interface; Bus interface; Arithmetic co-processor; Microcontrollers; Integrating microprocessor with interfacing chips.

Computer Architecture

Information representation; Measuring performance; Instructions and data access methods; operations and operands of computer hardware, representing instruction, addressing styles; Arithmetic Logic Unit (ALU) operations, floating point operations, designing ALU; Processor design: datapaths – single cycle and multicycle implementations; Control Unit design - hardware and microprogrammed; Hazards; Exceptions; Pipeline: pipelined datapath and control, superscalar and dynamic pipelining; Memory organization: cache, virtual memory, channels; DMA and Interrupts; Buses; Multiprocessors: types of multiprocessors, performance, single bus multiprocessors, multiprocessors connected by network, clusters.

Software Engineering and Information System Design

Concepts of Software Engineering, Software Engineering paradigms, Different phases of software System Development, Different types of information, qualities of information. Project Management Concepts, Software process and project Metrics, Software Project Planning, Risk Analysis and management, Project Scheduling and Tracking.

Analysis Concepts and principles: requirement analysis, Analysis modeling, data modeling. Design concepts and principles, Architectural design, User Interface design, Object Oriented software development and design: Iterative Development and the Unified Process. Sequential waterfall life cycles, Inception. Use case model for requirement writing, Elaboration using System Sequence Diagram, Domain Model. Visualizing concept classes. UML diagrams, Interaction and Collaboration Diagram for designing Software. Designing Objects with responsibilities. GRASP patterns with General Principles in assigning responsibilities: Information expert, Creator, Low Coupling and High Cohesion, Creating design class diagrams and mapping design to codes. Advanced GRASP patterns: Polymorphism, Pure Fabrication, Indirection, Project Variation. GoF Design Patterns: Adapter, Factory, Singleton, Strategy, Composite, Façade, and Observer.

Software Testing: White Box and Black Box testing. Basis Path Testing. Testing for specialized environment. Software testing strategies: Unit Testing, Integration Testing, Validation Testing, System Testing, Art of debugging.

Analysis of System Maintenance and upgrading: Software repair, downtime, error and faults, specification and correction, Maintenance cost models, documentation. Software Quality Assurance, Quality factors. Software quality measures. Cost impact of Software defects. Concepts of Software reliability, availability and safety. Function based metrics and bang metrics. Metrics for analysis and design model. Metrics for source code, testing and maintenance.

Database

Concepts of database systems; Models: Entity-Relationship model, Relational model; Relational algebra; SQL; Integrity constraint; Relational database design; File organization and retrieval, file indexing; Transaction manager; Concurrency controller; Recovery manager; Security system; Database administration; Advanced database management systems: distributed, multimedia, object-oriented, object-relational; Some applications using SQL.

Operating System

Operating System: its role in computer systems; Operating system concepts; Operating system structure; Process: process model and implementation, Inter-Process, Communication (IPC), classical IPC problems, process scheduling, multiprocessing and time-sharing; Memory management: swapping, paging, segmentation, virtual memory; Input/Output: hardware, software, disk, terminals, clocks; Deadlock: resource allocation and deadlock, deadlock detection, prevention and recovery; File Systems: files, directories, security, protection; Case study of some operating systems.

Computer Networks

Protocol hierarchies; Data link control: HDLC; DLL in Internet; DLL of ATM; LAN Protocols: Standards IEEE 802.*; Hubs, Bridges, and Switches, FDDI, Fast Ethernet; Routing algorithm; Congestion control; Internetworking, WAN; Fragmentation; Firewalls; IPV4, IPV6, ARP, RARP, Mobile IP, Network layer of ATM; Transport protocols; Transmission control protocol: connection management, transmission policy, congestion control, timer management; UDP; AAL of ATM; Network security: Cryptography, DES, IDEA, public key algorithm; Authentication; Digital signatures; Gigabit Ethernet; Domain Name System: Name servers; Email and its privacy; SNMP; HTTP; World Wide Web.

Artificial Intelligence

Introduction to old and new AI techniques; Knowledge representation; Propositional and first order logic, inference in first order logic; Frame problem; Search techniques in AI; Game playing; Planning; Probabilistic reasoning; Learning in symbolic and non-symbolic representation; Natural language processing. Introduction to expert system.

Digital System Design

Designing I/O system; I/O devices; Designing Microprocessor based system with interfacing chips; Programmable peripheral interface (interface to A/D and D/A converter); Keyboard/display interface; Programmable timer; Programmable interrupt controller, DMA controller; Design using MSI and LSI components; Design of memory subsystem using SRAM and DRAM; Design of various components of a computer: ALU, memory and control unit – hardwired and micro programmed; Microprocessor based designs; Computer BUS standards; Design special purpose controllers.

Basic Graph Theory

Graphs: simple graphs, digraphs, subgraphs, vertex-degrees, walks, paths and cycles; Trees, spanning trees in graphs, distance in graphs; Complementary graphs, cut-vertices, bridges and blocks, k-connected graphs; Euler tours, Hamiltonian cycles, Chinese Postman Problem, Traveling Salesman Problem; Chromatic number, chromatic polynomials, chromatic index, Vizing's theorem, planar graphs, perfect graphs.

Machine Learning

Introduction to machine learning; Learning algorithms: supervised, unsupervised, reinforcement, attribute based, neural network based, relational supervised and negative correlation; Genetic algorithm, genetic programming and evolutionary programming; Practical application of machine learning.

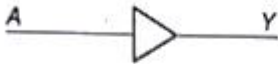
Computer Graphics

Graphics hardware: display devices, input devices etc; Basic raster graphics algorithms for drawing 2D primitives; Two-dimensional and three-dimensional viewing, clipping and transformations; Three-dimensional object representations: polygon surface, BSpline curves and surfaces, BSP trees, Octrees, Fractal-Geometry methods; Visible surface detection methods: Z-buffer method, BSP tree method, Ray casting method; Illumination models; Surface rendering methods: polygon rendering, ray tracing, terrain visualization with height mapping, modeling surface details with texture mapping; Color models; Computer animation.

Digital Logic Design

Important Topics :

- Combinational Circuit vs Sequential Circuit
- Simplifying the Equations by Using K-map
- SOP, POS/ SOM, POM
- Circuit Implementation by using X-OR, NAND, NOR gates.
- Decoder and Encoder Implementation
- Counter Design (Mod 6, Mod 12) using T-FF, Asynchronous, Synchronous Counter Design
- Truth Table to Equations
- MUX, DeMUX
- Race around Problem
- FF Summary
- SOP Implementation by MUX
- MUX tree design

Name	Graphic Symbol	Algebraic Function	Truth Table																		
Buffer		$Y = A$	<table><tr><th>Input A</th><th>Output Y</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	Input A	Output Y	0	0	1	1												
Input A	Output Y																				
0	0																				
1	1																				
Inverter		$Y = \bar{A}$	<table><tr><th>Input A</th><th>Output Y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	Input A	Output Y	0	1	1	0												
Input A	Output Y																				
0	1																				
1	0																				
AND		$Y = A.B$	<table><tr><th colspan="2">Input</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Input		Output	A	B	Y	0	0	0	0	1	0	1	0	0	1	1	1
Input		Output																			
A	B	Y																			
0	0	0																			
0	1	0																			
1	0	0																			
1	1	1																			

OR		$Y = A + B$	<table><tr><th colspan="2">Input</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Input		Output	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	1						
Input		Output																									
A	B	Y																									
0	0	0																									
0	1	1																									
1	0	1																									
1	1	1																									
NAND		$Y = \overline{A \cdot B}$	<table><tr><th colspan="3">Input</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>A.B</th><th>$Y = \overline{A \cdot B}$</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	Input			Output	A	B	A.B	$Y = \overline{A \cdot B}$	0	0	0	1	0	1	0	1	1	0	0	1	1	1	1	0
Input			Output																								
A	B	A.B	$Y = \overline{A \cdot B}$																								
0	0	0	1																								
0	1	0	1																								
1	0	0	1																								
1	1	1	0																								
NOR		$Y = \overline{A + B}$	<table><tr><th colspan="3">Input</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>A+B</th><th>$Y = \overline{A + B}$</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	Input			Output	A	B	A+B	$Y = \overline{A + B}$	0	0	0	1	0	1	1	0	1	0	1	0	1	1	1	0
Input			Output																								
A	B	A+B	$Y = \overline{A + B}$																								
0	0	0	1																								
0	1	1	0																								
1	0	1	0																								
1	1	1	0																								

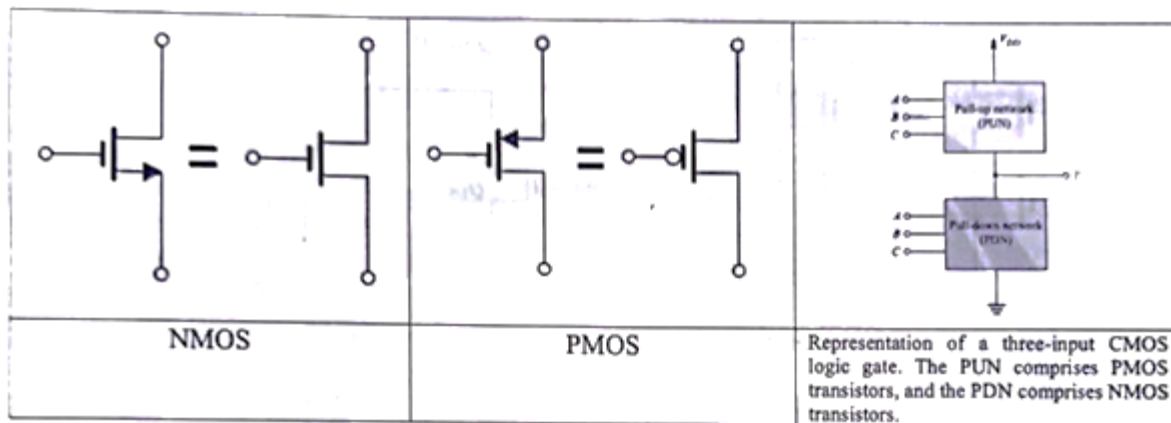
Name	Graphic Symbol	Algebraic Function	Truth Table																		
Exclusive-OR (X-OR)		$Y = A \oplus B$ $= A \bar{B} + \bar{A} B$	<table><tr><th colspan="2">Input</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>$Y = A \oplus B$ $= A \bar{B} + \bar{A} B$</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Input		Output	A	B	$Y = A \oplus B$ $= A \bar{B} + \bar{A} B$	0	0	0	0	1	1	1	0	1	1	1	0
Input		Output																			
A	B	$Y = A \oplus B$ $= A \bar{B} + \bar{A} B$																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	0																			
Exclusive-NOR (X-NOR)		$Y = A \odot B$ $= AB + \bar{A} \bar{B}$	<table><tr><th colspan="2">Input</th><th>Output</th></tr><tr><th>A</th><th>B</th><th>$Y = A \odot B$ $= AB + \bar{A} \bar{B}$</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Input		Output	A	B	$Y = A \odot B$ $= AB + \bar{A} \bar{B}$	0	0	1	0	1	0	1	0	0	1	1	1
Input		Output																			
A	B	$Y = A \odot B$ $= AB + \bar{A} \bar{B}$																			
0	0	1																			
0	1	0																			
1	0	0																			
1	1	1																			

Postulates and Theorems of Boolean Algebra

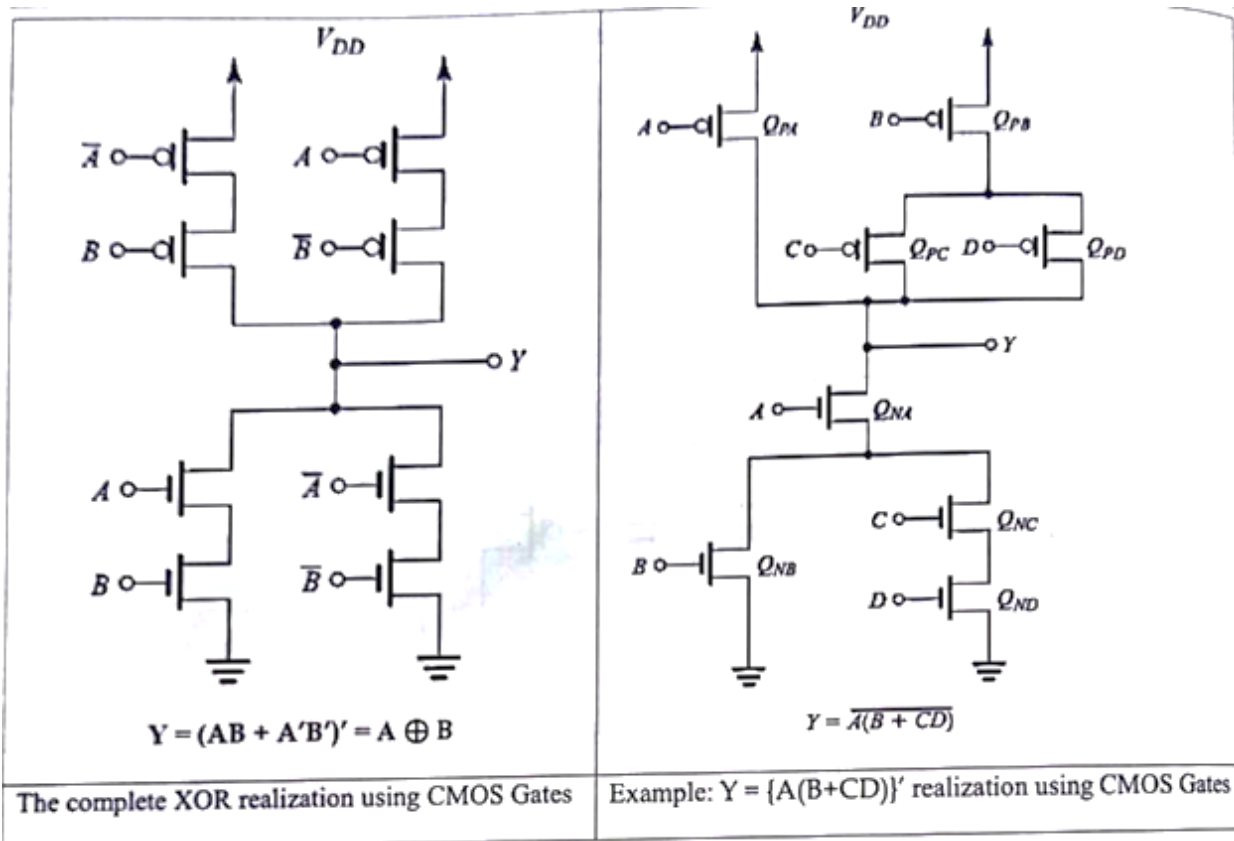
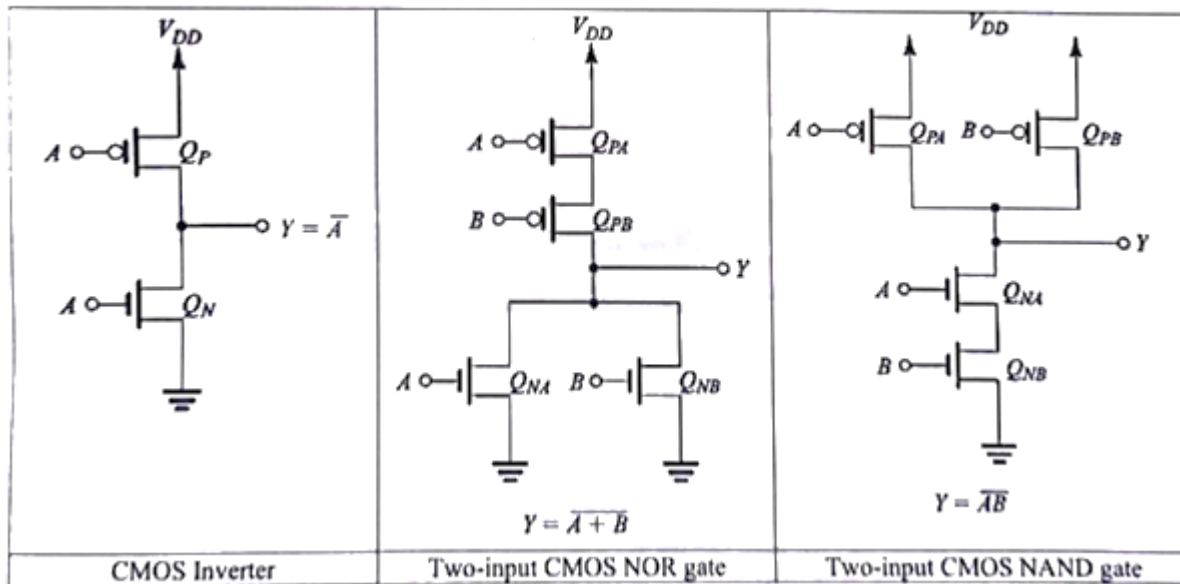
	$A + 0 = A$	$A.1 = A$
	$A + A' = 1$	$A.A' = 0$
	$A + A = A$	$A.A = A$
	$A + 1 = 1$	$A.0 = 0$
Involution	$(A')' = A$	
Commutative	$A + B = B + A$	$AB = BA$
Associative	$A + (B+C) = (A+B) + C$	$A(BC) = (AB)C$
Distributive	$A(B+C) = AB + AC$	$A + BC = (A+B)(A+C)$
De-Morgan's Law	$(A+B)' = A'B'$	$(AB)' = A' + B'$
Absorption	$A + BC = A + B$	$A(A+B) = A$

CMOS logic circuits

A CMOS logic circuit is in effect an extension, or a generalization, of the CMOS inverter. The inverter consists of an NMOS pull-down transistor, and a PMOS pull-up transistor, operated by the input voltage in a complementary fashion. The CMOS logic gate consists of two networks: the pull-down network (PDN) constructed of NMOS transistors, and the pull-up network (PUN).



Important CMOS Gates



1 Implement AND, OR, NOT Gate using universal NAND Gate.

Solution:

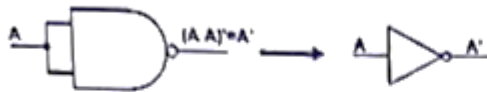


Figure: NOT gate using NAND Gate

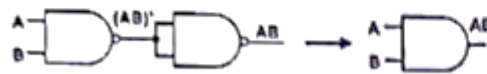


Figure: AND gate using NAND Gate

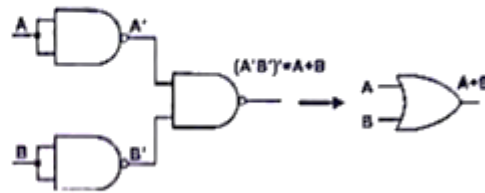
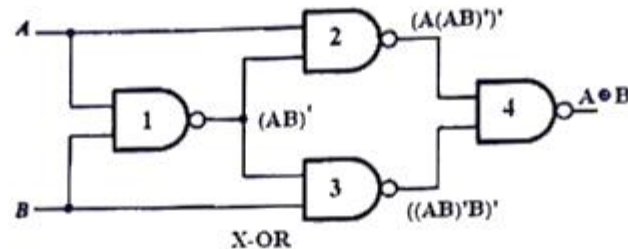


Figure: OR gate using NAND Gate

5 Draw the expression using NAND gate: $AB' + A'B$ BUET M.Sc.-15

Solution:



Explanation:

Gate No.	Inputs	Output
1	A, B	$(AB)'$
2	A, $(AB)'$	$(A(AB)')'$
3	$(AB)'$, B	$(B(AB)')'$
4	$(A(AB)')'$, $(B(AB)')'$	$A'B + AB'$

Now the output from gate no. 4 is the overall output of the configuration.

$$\begin{aligned}
 Y &= ((A(AB)')'(B(AB)')')' \\
 &= (A(AB)')'' + (B(AB)')'' \\
 &= (A(AB)') + (B(AB)') \\
 &= (A(A' + B')) + (B(A' + B')) \\
 &= (AA' + AB') + (BA' + BB') \\
 &= (0 + AB' + BA' + 0) \\
 &= AB' + BA' \\
 \Rightarrow Y &= AB' + A'B
 \end{aligned}$$

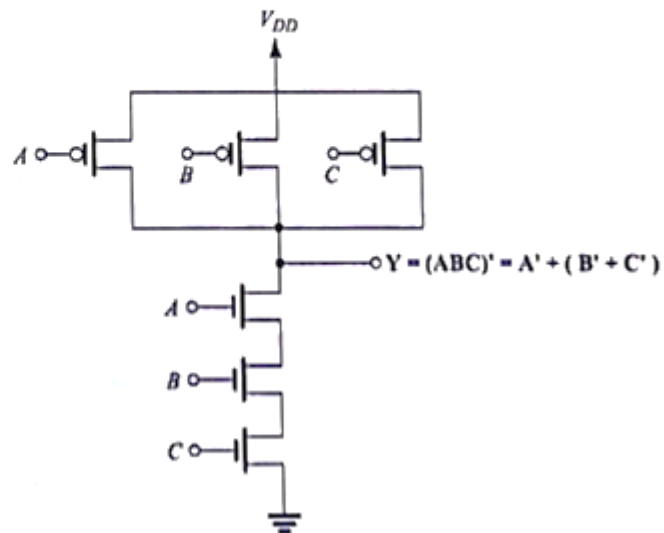
12	Draw the CMOS complex gate logic circuit function on $Y = A' + (B' + C')$	BUET M.Sc.-18, BUET MSC-14
----	--	-------------------------------

Solution:

$$Y = A' + (B' + C')$$

$$\begin{aligned} \therefore Y' &= \{A' + (B' + C')\}' \\ &= (A')' \cdot (B' + C')' \\ &= (A)' \cdot \{(B')' \cdot (C')'\} \\ &= A B C \end{aligned}$$

$$\therefore Y = (Y')' = (A B C)'$$



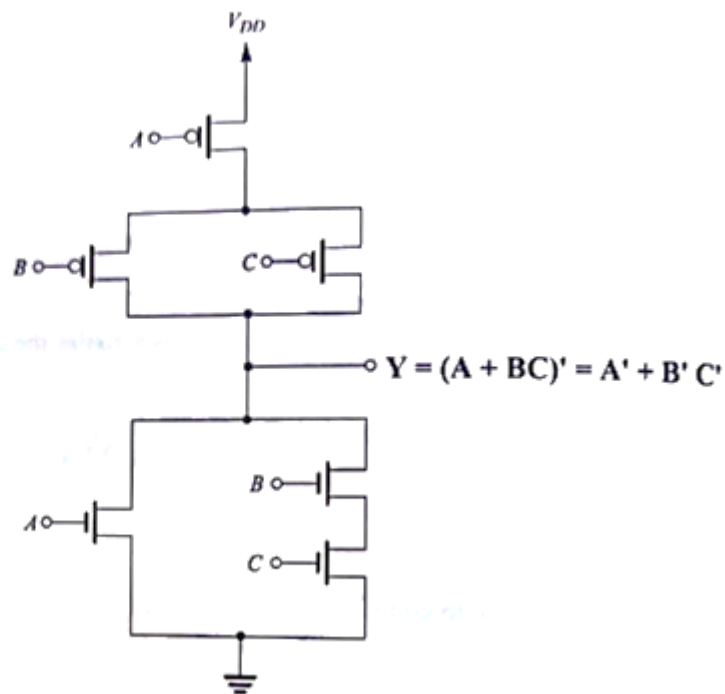
13	Draw the CMOS complex gate logic circuit function on $Y = A' + B' C'$	BUET M.Sc.-17
----	---	------------------

Solution:

$$Y = A' + B' C'$$

$$\begin{aligned} \therefore Y' &= (A' + B' C')' \\ &= (A')' (B' C')' \\ &= A \{(B')' + (C')'\} \\ &= A + BC \end{aligned}$$

$$\therefore Y = (Y')' = (A + BC)'$$



Basic k-Map

A \ B	0	1
0	0	1
1	2	3

(a) Two Variables

A \ BC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

(b) Three Variables

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

(c) Four Variables

Find SOP from the following K-Map.

AB \ CD	00	01	11	10
00	d	1	1	d
01	0	d	1	0
11	0	0	1	0
10	0	0	1	0

This problem can be solved in two ways-

Way-1: Part (a)

AB \ CD	00	01	11	10
00	d	1	1	d
01	0	d	1	0
11	0	0	1	0
10	0	0	1	0

$\underbrace{\hspace{10em}}_D$

$$SOP = CD + A'B'$$

Ans.

Way-2 : Part (b)

AB \ CD	00	01	11	10
00	d	1	1	d
01	0	d	1	0
11	0	0	1	0
10	0	0	1	0

$\underbrace{\hspace{10em}}_D$

$$SOP = CD + A'B$$

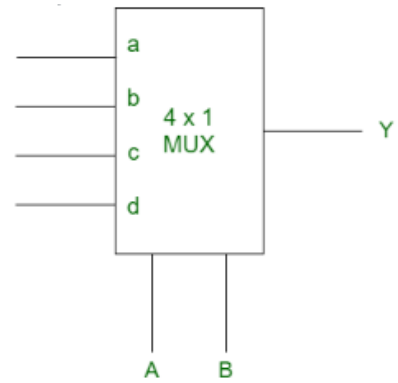
Ans.

Implementation of SOP function using multiplexer

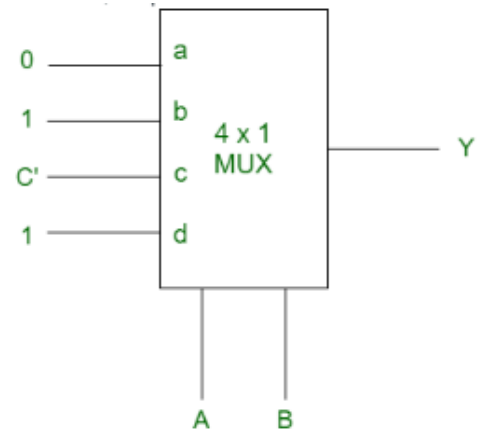
$$f(A, B, C) = m(0, 1, 4, 6, 7)$$

Truth Table

	A	B	C	Y
0	0	0	0	1
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

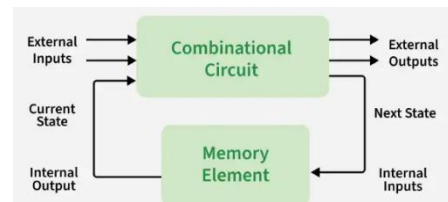
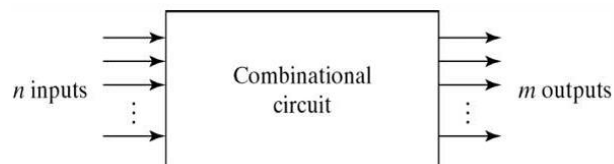


	a	b	c	d
C'	0	2	4	6
C	1	3	5	7
	1	0	C'	1



Combinational Circuit vs Sequential Circuit

Combinational Circuit	Sequential Circuit
Output depends only on the current inputs.	Output depends on both current inputs and past states (memory).
Does not require memory elements.	Requires memory elements like flip-flops or latches.
Output is immediate, based on input changes.	Output is dependent on clock pulses and previous states.
No clock signal required.	Requires a clock signal to synchronize state changes.
Simpler design without the need for memory.	More complex due to memory and clock management.
Faster, as outputs change instantly with inputs.	Slower due to dependency on clock cycles and past states.
Performs basic logical operations without sequence dependency.	Performs operations that require sequences or timed events.
<u>Adders</u> , <u>Subtractors</u> , <u>Multiplexers</u> , <u>Encoders</u> .	Counters, <u>Shift Registers</u> , <u>Flip-Flops</u> , State Machines.
Generally lower power consumption.	Higher power consumption due to memory and clock circuitry.
Used in tasks requiring direct logical operations (e.g., arithmetic).	Used in applications involving sequential operations (e.g., counters, registers).



Encoders and Decoders in Digital Logic

Binary code of N digits can be used to store 2^N distinct elements of coded information. This is what encoders and decoders are used for.

Encoders convert 2^N lines of input into a code of N bits and **Decoders** decode the N bits into 2^N lines.

Diagram of an 8x3 Encoder:

- Inputs: D0, D1, D2, D3, D4, D5, D6, D7
- Block: 8x3 Encoder
- Outputs: X, Y, Z
- Text: of a N bit As

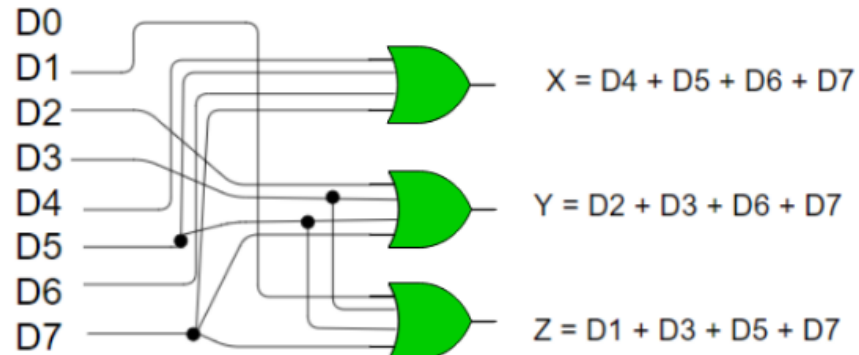
[illegible]

$$X = D4 + D5 + D6 + D7$$

$$Y = D2 + D3 + D6 + D7$$

$$Z = D1 + D3 + D5 + D7$$

Hence, the encoder can be realised with OR gates as follows:



Truth Table -

2. Decoders -

A decoder is a combinational circuit that performs the reverse operation of an encoder, converting n input lines into 2^n output lines, with only one output active each input combination.

Let's take an example of 3-to-8 line decoder.

X	Y	Z	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

for

Implementation -

D0 is high when $X = 0$, $Y = 0$ and $Z = 0$. Hence,

$$D_0 = X' Y' Z'$$

Similarly,

$$D_1 = X' Y' Z$$

$$D_2 = X' Y Z'$$

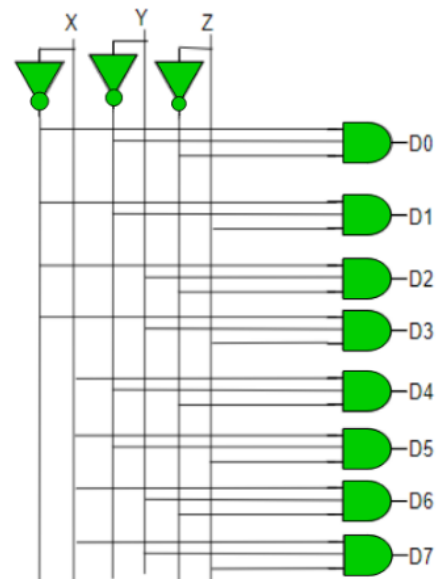
$$D_3 = X' Y Z$$

$$D_4 = X Y' Z'$$

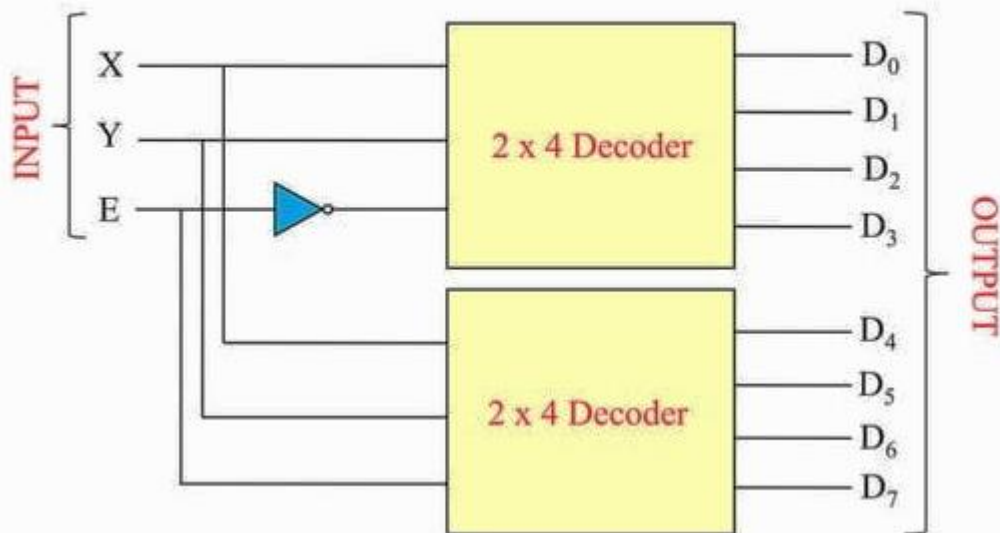
$$D_5 = X Y' Z$$

$$D_6 = X Y Z'$$

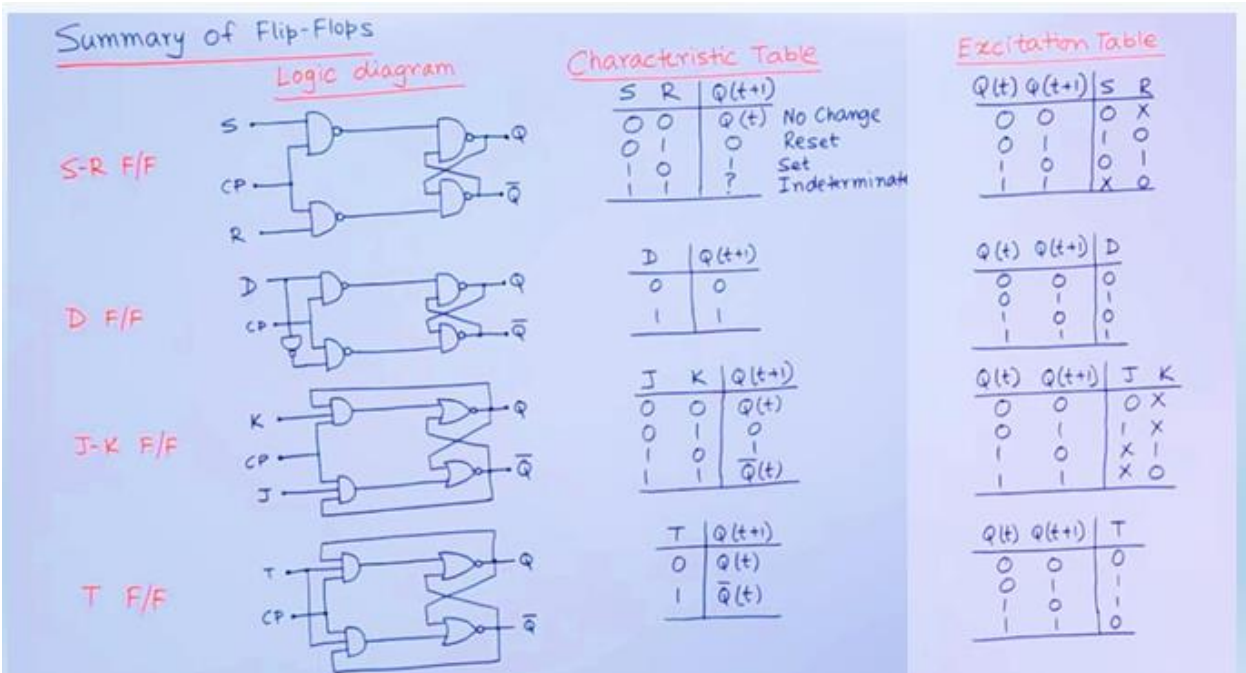
$$D_7 = X Y Z$$



3 x 8 Decoder From 2 x 4 Decoder:



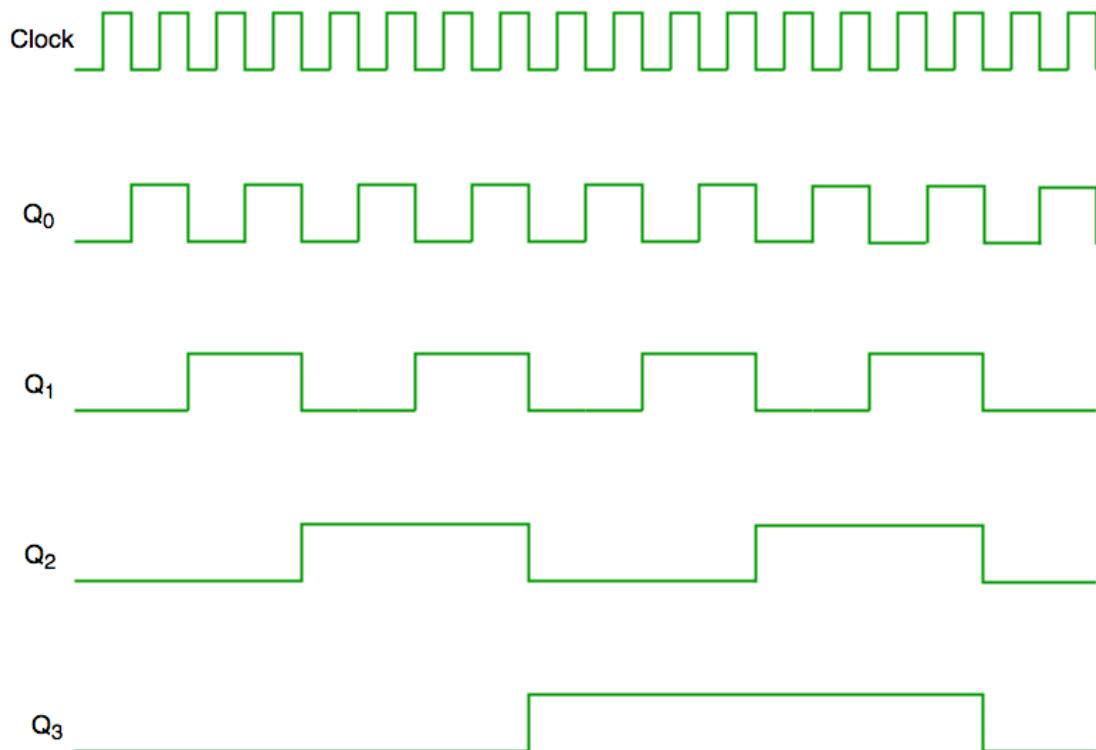
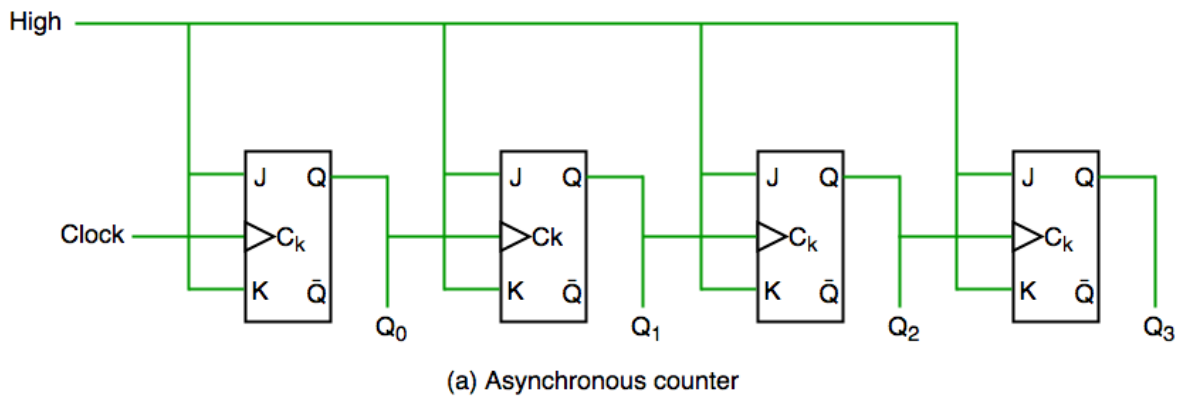
Flip-Flop	Latch
Flip-flop is a bistable device i.e., it has two stable states that are represented as 0 and 1.	Latch is also a bistable device whose states are also represented as 0 and 1.
It checks the inputs but changes the output only at times defined by the clock signal or any other control signal.	It checks the inputs continuously and responds to the changes in inputs immediately.
It is a edge triggered device.	It is a level triggered device.
Gates like <u>NOR</u> , <u>NOT</u> , <u>AND</u> , <u>NAND</u> are building blocks of flip flops.	These are also made up of gates.
They are classified into asynchronous or synchronous flipflops.	There is no such classification in latches
It forms the building blocks of many sequential circuits like <u>counters</u> .	These can be used for the designing of sequential circuits but are not generally preferred.
Flip-flop always have a clock signal	Latches doesn't have a clock signal
Flip-flop can be build from Latches	Latches can be build from gates
ex: <u>D Flip-flop</u> , <u>JK Flip-flop</u>	ex: <u>SR Latch</u> , <u>D Latch</u>



Counter Classification

1. Asynchronous Counter

In an asynchronous counter, there is no common (universal) clock for all flip-flops; only the first flip-flop is driven by the main clock, and each subsequent flip-flop is triggered by the output of the previous one, causing the count to ripple through the circuit.

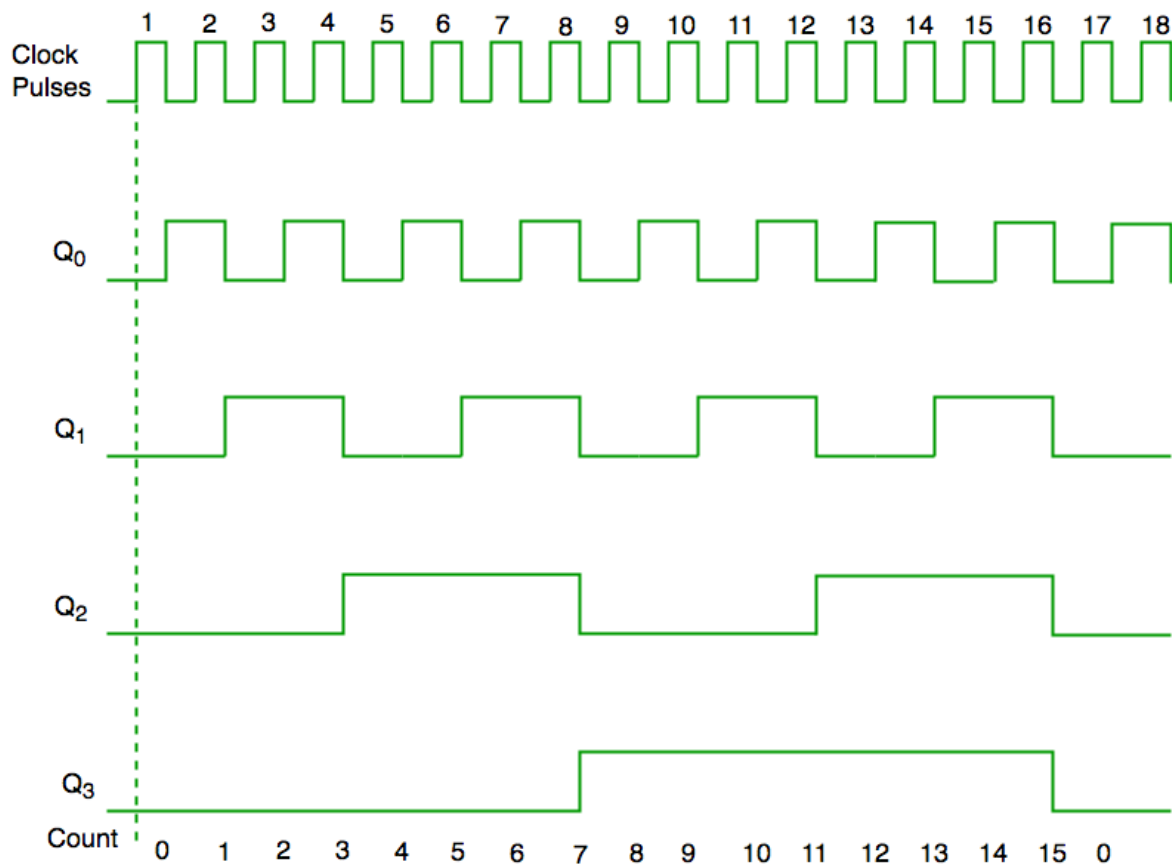
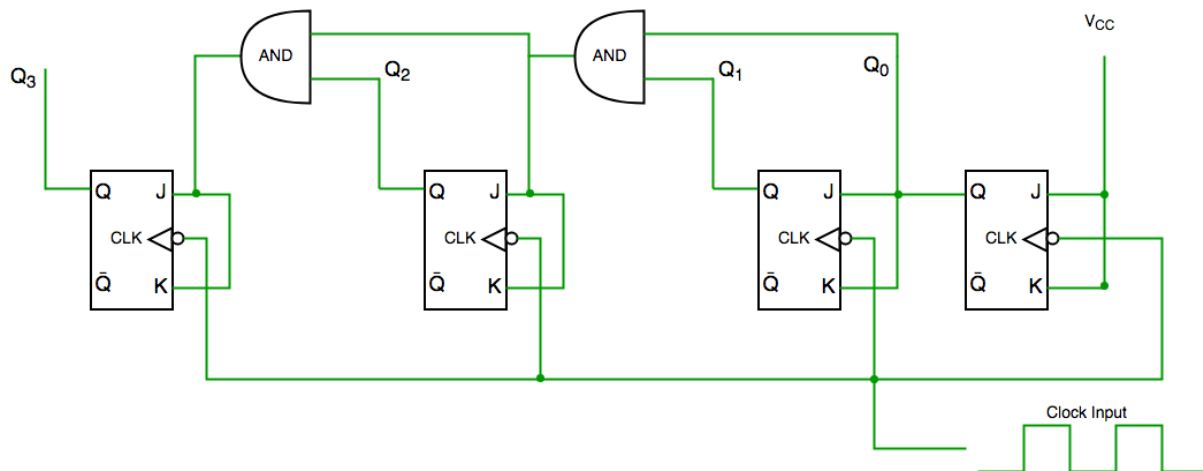


(b) Timing Diagram

2. Synchronous Counter

Unlike an asynchronous counter, a synchronous counter uses a single global clock that drives all flip-flops simultaneously, so all outputs change in parallel. Its main advantage is that it can operate at

higher frequencies because it does not suffer from cumulative delay, as the same clock is applied to every flip-flop. Hence, it is also called a parallel counter.



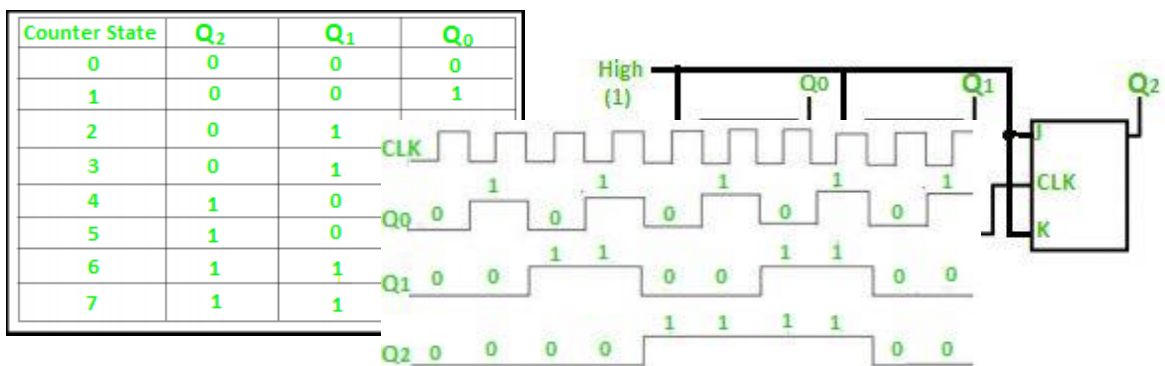
Ripple Counter

A ripple counter is an asynchronous counter made by cascading flip-flops, where the output of one flip-flop drives the clock of the next. Only the first flip-flop receives the external clock; others are triggered by the previous stage's output, causing a "ripple" effect.

- Asynchronous operation.
- Flip-flops operate in toggle mode.
- Only one flip-flop receives the external clock (LSB).
- Can be configured as up, down, or up/down counter.

An n-bit ripple counter has 2^n states and is also called a MOD-n counter. Counting sequences repeat after reaching the maximum or minimum count:

- Up counter: $000 \rightarrow 001 \rightarrow \dots \rightarrow 111 \rightarrow 000\dots$
- Down counter: $111 \rightarrow 110 \rightarrow \dots \rightarrow 000 \rightarrow 111\dots$

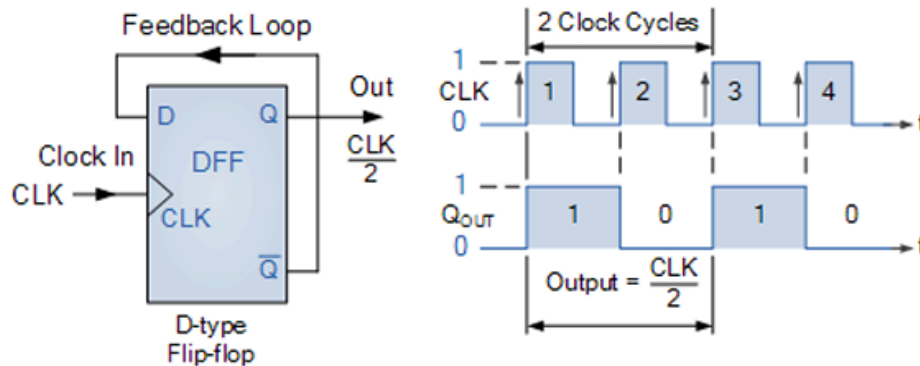


S.NO	PLA	PAL
1.	PLA stands for Programmable Logic Array.	While PAL stands for Programmable Array Logic.
2.	PLA speed is lower than PAL.	While PAL's speed is higher than PLA.
3.	The complexity of PLA is high.	While PAL's complexity is less.
4.	The cost of PLA is also high.	While the cost of PAL is low.
5.	Programmable Logic Array is less available.	While Programmable Array Logic is more available than Programmable Logic Array.
6.	It is less used than PAL.	While it is more used than PLA.

10. How can we Convert a 100 MHz clock to 50 MHz and 25 MHz by only using D flip-flops?
[BUET]

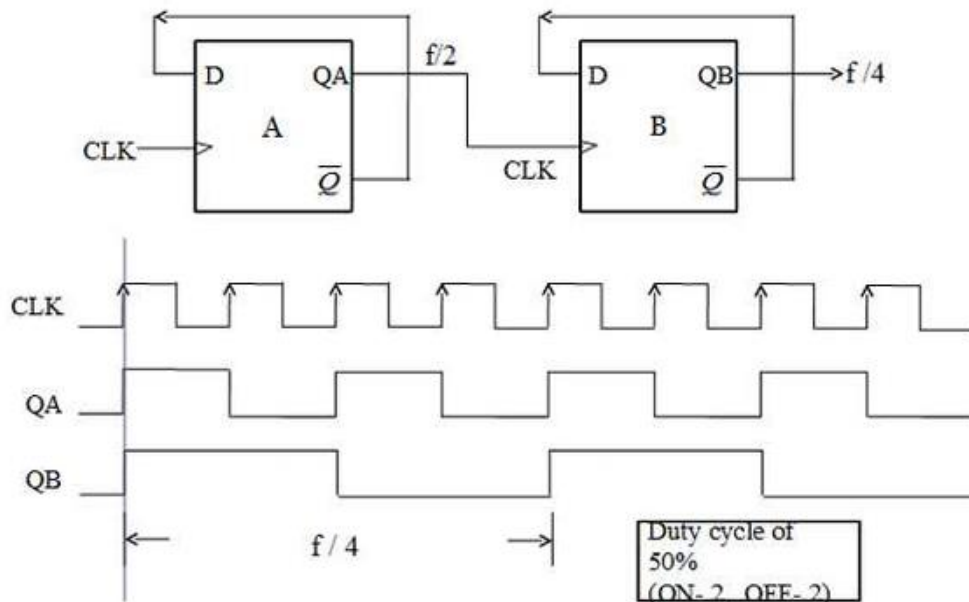
<https://www.quora.com/How-can-we-convert-a-100-MHz-clock-to-50-MHz-and-25-MHz-by-only-using-D-flip-flops>

D flip flop may be used as divide by 2 counter by connecting $\bar{Q}$ back to D input as shown below.



Clearly by this we will be able to get 50 MHz from input of 100 MHz and if we want again 25 MHz out of it cascade one more stage as shown below.

Frequency Divide by 4 ($f/4$)



So if clk $f_{in} = 100$ MHz, QA output frequency = 50 MHz and QB output frequency = 25 MHz.

--